

Rendu individuel

Zaafir Mougammadou Zaccaria
Infrastructure cloud & déploiement applicatif

Auteur	Zaafir Mougammadou Zaccaria
Rôle	Infrastructure cloud & déploiement applicatif
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Infrastructure as code Terraform, application FastAPI/React, déploiement

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

Au sein de l'équipe, j'ai assumé la responsabilité de deux briques indissociables : l'infrastructure Azure décrite sous forme de code, et l'application elle-même, de son développement jusqu'à son déploiement sur le cluster. Mon fil conducteur a été un principe simple mais exigeant : partir d'un dépôt vide, il devait être possible de reconstruire l'intégralité de l'environnement — réseau, cluster, base de données, application — de façon entièrement automatique et strictement reproductible, sans qu'aucune étape ne repose sur une manipulation manuelle mémorisée par un membre de l'équipe.

Ce positionnement me plaçait à la charnière entre le développement et l'exploitation, ce qui correspond exactement à l'esprit DevOps : je devais comprendre les besoins fonctionnels du cabinet pour concevoir l'application, tout en anticipant les contraintes d'hébergement, de sécurité et de coût qui conditionnent sa mise en production. Cette double casquette a nourri l'ensemble de mes choix techniques.

2. Ma contribution détaillée

2.1 Conception de l'infrastructure Terraform

J'ai écrit les modules Terraform qui décrivent l'ensemble des ressources Azure du projet : le groupe de ressources qui les regroupe, le cluster Kubernetes managé (AKS), le registre d'images Azure Container Registry, le serveur PostgreSQL Flexible, ainsi que la configuration réseau et le budget de surveillance des coûts. Chaque ressource est paramétrée par des variables afin de pouvoir ajuster la région, la taille des machines ou l'activation de certains composants sans modifier le code.

Un soin particulier a été porté à l'**idempotence** de cette description : appliquée plusieurs fois, elle doit converger vers le même état sans effet de bord. C'est cette propriété qui permet à la chaîne d'intégration de rejouer le déploiement en toute confiance, et qui garantit qu'un membre de l'équipe obtiendra exactement la même infrastructure qu'un autre. J'ai également veillé à ce que les dépendances entre ressources soient correctement exprimées, afin que Terraform les crée dans le bon ordre (par exemple, le registre et le rôle d'accès avant le cluster qui doit tirer les images).

La gestion de l'état Terraform a fait l'objet d'une attention spécifique : plutôt qu'un état local fragile, nous avons opté pour un état distant stocké dans un compte de stockage Azure, ce qui permet le travail à plusieurs et évite les conflits. J'ai conçu la configuration de sorte que ce stockage soit créé automatiquement s'il n'existe pas, supprimant toute étape préalable manuelle.

2.2 Développement de l'application

J'ai développé le cœur applicatif avec le framework FastAPI, en Python. L'API expose les ressources métier du cabinet : les patients, les appareils auditifs qui leur sont associés et les rendez-vous de suivi. J'ai structuré le code en couches claires — modèles de données, schémas de validation, logique d'accès aux données et routes HTTP — afin qu'il reste lisible et maintenable. La validation des entrées est confiée à Pydantic, ce qui garantit qu'aucune donnée mal formée n'atteint la base.

J'ai doté l'API de **sondes de santé** distinctes : une sonde de vivacité qui confirme que le service répond, et une sonde de disponibilité qui vérifie en outre que la base de données est joignable. Cette distinction est essentielle pour Kubernetes, qui s'appuie sur ces sondes pour décider de redémarrer un conteneur ou de lui envoyer du trafic. J'ai également exposé un point de terminaison de métriques, consommé par la supervision mise en place par un autre membre de l'équipe.

Côté interface, j'ai réalisé un frontend React servi par un serveur nginx, permettant au cabinet de consulter et de gérer ses patients de façon simple. L'interface consomme l'API via un chemin unique, ce qui simplifie le routage en production où frontend et backend sont exposés derrière le même point d'entrée.

2.3 Conteneurisation et déploiement

J'ai conteneurisé les deux composants au moyen d'images Docker construites en plusieurs étapes (multi-stage), afin de séparer la phase de compilation de l'image finale et d'obtenir des images légères. Ces images s'exécutent avec un utilisateur non privilégié, conformément aux bonnes pratiques de sécurité, un point sur lequel j'ai collaboré avec le responsable sécurité de l'équipe.

Le déploiement sur le cluster s'appuie sur le chart Helm conçu avec le responsable Kubernetes : j'ai veillé à ce que l'application reçoive sa configuration (notamment la chaîne de connexion à la base) par l'intermédiaire d'un secret Kubernetes injecté au moment du déploiement, et jamais écrit en clair dans le dépôt. J'ai enfin coordonné l'enchaînement complet du déploiement de bout en bout et vérifié, après chaque livraison, le bon fonctionnement de l'application, de la base et de l'interface.

3. Choix techniques et justifications

Le choix de FastAPI s'explique par sa rapidité de mise en œuvre, sa documentation automatique de l'API et sa robustesse grâce à la validation typée, autant d'atouts pour un projet à durée contrainte. PostgreSQL s'est imposé comme base relationnelle éprouvée, disponible en version managée sur Azure, ce qui nous décharge de son exploitation (sauvegardes, correctifs) tout en restant peu coûteux au palier le plus bas.

Concernant l'hébergement de la base, nous avons retenu un accès public restreint par un pare-feu n'autorisant que les ressources internes à Azure, avec chiffrement TLS obligatoire. C'est un compromis assumé : un réseau entièrement privé aurait été plus sûr, mais aurait ajouté une complexité (réseau virtuel dédié, résolution DNS privée) difficilement justifiable pour un MVP au budget serré. J'ai documenté ce compromis pour qu'il soit clairement identifié comme un axe d'amélioration.

4. Difficultés rencontrées et solutions apportées

La principale difficulté a été d'**industrialiser le provisionnement** pour qu'il soit réellement rejouable par la chaîne d'intégration. Les premières exécutions ont révélé des situations délicates : un déploiement interrompu laissait l'état Terraform verrouillé, empêchant toute reprise. En collaboration avec le responsable CI/CD, nous avons ajouté un mécanisme libérant automatiquement ce verrou résiduel avant chaque application, rendant la chaîne robuste.

Une seconde difficulté a concerné la cohérence entre la version de l'application déployée et la configuration attendue : j'ai résolu ce point en faisant en sorte que chaque déploiement utilise une étiquette d'image unique, ce qui force le remplacement propre des conteneurs et garantit que la nouvelle configuration est bien prise en compte. Enfin, un souci subtil de compatibilité entre le format du mot de passe de la base et l'outil de sauvegarde m'a conduit à générer un mot de passe strictement alphanumérique, éliminant toute une classe d'erreurs d'interprétation d'URL.

5. Perspectives d'évolution

À court terme, je remplacerais l'authentification de la chaîne d'intégration par une fédération d'identité OIDC entre GitHub et Azure. Ce mécanisme supprime tout mot de passe stocké au profit de jetons éphémères, ce qui constitue l'état de l'art et lèverait la contrainte actuelle d'un compte sans double authentification.

Je ferais également évoluer la base vers un accès strictement privé, sans exposition publique, en plaçant l'application et la base dans un réseau virtuel commun avec des points de terminaison privés. Cette évolution renforcerait significativement l'isolation des données de santé, conformément aux attentes d'un véritable contexte de production.

À plus long terme, si le périmètre fonctionnel s'élargissait (facturation, tiers payant, notifications), l'API monolithique pourrait être découpée en services plus fins et indépendamment déployables. J'introduirais aussi un outil de migrations de schéma versionnées, mieux adapté qu'une création au démarrage pour faire évoluer la base sans risque en production.

6. Analyse critique des limites

La limite la plus notable de mon périmètre tient au compromis réseau évoqué plus haut : l'accès public restreint reste moins satisfaisant qu'une isolation complète. De même, la création du schéma au démarrage de l'application est pratique pour un MVP mais inadaptée à une exploitation durable, où toute modification de structure doit être maîtrisée et réversible.

Sur le plan des performances, le choix de machines burstable plafonne la capacité sous une charge soutenue et prolongée ; ce dimensionnement, pertinent pour une démonstration, devrait être réévalué pour un usage réel du cabinet. Enfin, la couverture de tests automatisés de l'application, bien que présente, gagnerait à être étendue aux scénarios d'erreur et aux cas limites.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Déployer l'environnement

Le déploiement se résume, pour l'utilisateur, à trois étapes. D'abord, renseigner les identifiants Azure dans les secrets du dépôt (identifiant, mot de passe, tenant, abonnement). Ensuite, lancer le workflow de déploiement depuis l'onglet Actions, ou pousser du code sur la branche principale. Enfin, récupérer l'adresse d'accès affichée à la fin de l'exécution. L'infrastructure complète est alors provisionnée puis l'application déployée, sans autre intervention.

7.2 Utiliser l'application

L'application est accessible à l'adresse publique de l'Ingress. L'interface web permet de créer, consulter et supprimer des patients ; la documentation interactive de l'API, utile pour les tests ou l'intégration, est publiée sous le chemin « /api/docs ». Les sondes « /healthz » et « /readyz » permettent de vérifier rapidement, en ligne de commande, que le service et sa base répondent.

7.3 Reconstruire ou détruire

L'ensemble de l'infrastructure peut être reconstruit à l'identique par une seule commande, ou détruit en choisissant l'action de destruction du workflow — ce qui stoppe toute facturation. Cette reproductibilité est la garantie que le projet est duplicable et réutilisable, comme l'exige le cahier des charges.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de passer d'une logique de « code qui fonctionne sur ma machine » à une logique d'infrastructure et de déploiement entièrement automatisés et reproductibles. Cela m'a demandé de raisonner en permanence sur les cas d'échec et les états intermédiaires, et d'accepter plusieurs itérations avant d'obtenir une chaîne réellement fiable.

Forces

Je retiens comme force ma capacité à avoir une vision d'ensemble reliant le développement applicatif et son hébergement, ce qui m'a permis de concevoir une application pensée dès le départ pour être déployée et exploitée. J'ai également su collaborer étroitement avec les responsables CI/CD et Kubernetes pour résoudre des problèmes situés à la frontière de nos périmètres.

Faiblesses

J'ai parfois eu tendance à vouloir déployer l'ensemble en une seule fois plutôt qu'à valider par petits incréments testés, ce qui a occasionné des allers-retours. Par ailleurs, mes compétences en sécurité réseau cloud, sollicitées par le choix d'hébergement de la base, restent à approfondir.

Compétences développées

Ce projet m'a permis de consolider ma maîtrise de Terraform et de la conception d'infrastructure Azure, de la conteneurisation Docker et du déploiement via Helm, ainsi que du développement d'une API moderne intégrée à une base de données. J'ai surtout acquis une compréhension concrète de l'articulation entre code, infrastructure et exploitation.

Axes d'amélioration personnels

Pour de futurs projets, je souhaite adopter une approche plus incrémentale et systématiquement testée des changements d'infrastructure, et renforcer mes connaissances en réseau cloud (réseaux virtuels, points de terminaison privés, pare-feux applicatifs) afin de concevoir des architectures plus sûres dès la première version.